

the padlock icon in your browser URL bar. Always enforce modern versions:

Version	Status	Security level
TLS 1.0 & 1.1	Deprecated	Broken; do not use
TLS 1.2	Acceptable	Safe only if configured with strong ciphers
TLS 1.3	Preferred	Faster, highly secure, and removes outdated options.

Top open source tools for data in transit are:

- **Let's Encrypt:** An automated, non-profit certificate authority providing free SSL/TLS certificates to secure websites.
- **cert-manager:** Automates the issuance and renewal of Let's Encrypt certificates within Kubernetes environments.
- **Istio/Linkerd:** Service meshes that automatically encrypt internal microservice communications using mutual TLS (mTLS), requiring both sides to verify identity.
- **OpenSSL/LibreSSL:** Foundational cryptographic libraries that power internet protocols, web servers, and VPNs.
- **WireGuard:** A fast, lightweight, and modern VPN protocol built into the Linux kernel for secure remote access.

Building an integrated security defence

Integrating encryption for data at rest (stored data) and data in transit (moving data) is about creating a continuous chain of custody. If you only secure data while it travels, it sits exposed on your servers. If you only secure it on your servers, it can be intercepted during transmission.

A seamless integration ensures that data is encrypted at every boundary and that the transition between states happens safely in secure memory.

The core lifecycle: Data in motion vs data at static

To integrate both concepts, you must understand the exact moments where data switches states. The critical security vulnerability occurs at the intersections—the precise moment data is decrypted from the network to be processed or decrypted from the disk to be transmitted.

Here is how data flows through a securely integrated pipeline:

- **In transit (the tunnel):** Data moves from a client (e.g., a browser or mobile app) encrypted via TLS 1.3. At this stage, network routers and internet service providers can only see scrambled ciphertext.
- **The handoff (in-memory processing):** The data arrives at the destination server. The server terminates the TLS

connection, decrypting the data into the server's RAM. Crucially, the data should never touch the server's hard drive in this plaintext state.

- **At rest (the vault):** Before writing the incoming data to a permanent database or storage disk, the application immediately encrypts it using a symmetric algorithm like AES-256. Once written to disk, it is secured 'at rest'.

Architecture patterns for integration

In real-world engineering, this integration is achieved using specific architecture patterns depending on the environment.

Pattern A: The secure web application setup: In a standard cloud or web application, the integration acts like a relay race where the 'data baton' is always protected.

- **Ingress:** A user submits sensitive data. It travels over the internet secured by TLS 1.3 (transit).
- **Termination:** The web server (e.g., Nginx) terminates the TLS connection and hands the plaintext data directly to the application layer inside system memory.
- **Application encryption:** The application immediately encrypts the data using a unique key before passing it to the database.
- **Storage:** The database writes the encrypted block to a hard drive protected by full-disk encryption like LUKS or VeraCrypt (rest).

Pattern B: Zero-knowledge cloud storage: For tools like Cryptomator, the integration is completely shifted to the client-side to protect against untrusted cloud providers:

- You create a file on your local machine, which is protected by full-disk encryption (rest - layer 1).
- Cryptomator encrypts the specific file individually using your private passphrase (rest - layer 2).
- The encrypted file is synchronised to the cloud over an HTTPS/TLS connection (transit).
- The cloud provider stores the file. Even if the cloud provider's servers are breached or they look at your storage, they only see the pre-encrypted file.

The anchor of integration: Unified key management

The biggest operational challenge in combining both forms of encryption is key management. If you use separate, isolated systems to manage your TLS certificates (transit) and your disk/database keys (rest), the system becomes fragile and prone to human error. Integrated systems use a centralised secrets manager, such as HashiCorp Vault, to act as a single source of truth (Figure 1).

By centralising keys, an organisation can enforce unified security policies. For example, if a server is compromised, the centralised vault can instantly revoke both the network TLS certificates and the database access keys simultaneously, isolating the breach.